

AP Computer Science A

Course Syllabus • 2026–2027

Instructor	Mr. Alisher Farhadi
Email	afarhadi_te@flatbush.org
Meeting time	3:50 – 4:50 PM
Meeting days	Monday–Thursday rotation
Course dates	September 9, 2026 – March 31, 2027
Total sessions	80
Language	Java
AP Exam	AP Computer Science A, May 2027

1. Course Description

This course is a full AP Computer Science A curriculum built around one idea: **you are not learning about programming, you are working as a software developer.**

Every unit teaches Java concepts required by the College Board's AP Computer Science A Course and Exam Description. But every unit also asks you to do what professional developers do — read a specification, write code in a real IDE, commit it to version control, document it in a README, review a classmate's work, and ship a feature into an application you own.

By March you will have:

- Written and debugged several hundred lines of Java across ~30 graded assignments
- Built and shipped **one original desktop application with a graphical interface**, designed by you
- Maintained a public GitHub portfolio of every piece of code you wrote this year
- Practiced against real AP Free Response Questions and multiple-choice sets

The AP exam is one outcome of this course. The GitHub profile you walk away with is the other, and for many of you it will matter more.

2. What Computer Science Actually Looks Like

Roughly every third session ends with a **Field Note** — a short segment connecting the thing we just learned to how it is used in industry:

Concept in class	Where it lives in the real world
Classes and objects	Every mobile app models users, posts, and payments as objects
Methods and parameters	Every API endpoint you have ever called is a method with parameters
Loops and arrays	Netflix ranking your homepage; a bank reconciling transactions
2D arrays	Image filters, spreadsheets, game boards, medical imaging
Searching and sorting	Autocomplete, database indexes, ride-matching
Version control (Git)	Every professional codebase on earth
Code review	How every line of code at Google, Meta, and your bank gets approved
Documentation	The difference between code that survives and code that gets deleted

3. Platforms and Tools

You must create accounts on all of these during Week 1. All are free.

Platform	Purpose	Account needed
Google Classroom	Assignment posting, due dates, grades, announcements	School Google account
GitHub	All code and written work is submitted here	Free personal account
Replit	Cloud Java IDE — where you write and run code	Free account, sign in with GitHub
Code.org CSA	Supplemental videos, Neighborhood/Theater labs	Free, join code given in class
AP Classroom	College Board practice questions and progress checks	Join code given in class

Bring every day: a school-issued or personal laptop, charged. Tablets and phones cannot run the tools in this course.

A note on account names. Your GitHub username becomes part of a public portfolio you may show to a college or an employer. Choose accordingly. `ayelet-cohen-dev` ages well. `xxsleepy_gamerXx` does not.

4. Course Structure — The Nine Units

The College Board organizes AP CSA into four large units. We break those into nine teachable blocks that fit our 45-minute period.

Unit	Title	Sessions	Dates	AP CED coverage
0	Launch: Developer Setup & Version Control	2	Sep 9 – Sep 10	Practice 5
1	Java Foundations: Types, Expressions, Output	10	Sep 15 – Oct 8	1.1–1.6, 1.8
2	Using Objects, Strings & the Math Class	8	Oct 12 – Oct 22	1.7, 1.9–1.15
3	Selection: Conditionals & Boolean Logic	9	Oct 26 – Nov 10	2.1–2.6
4	Iteration: Loops & Algorithms	10	Nov 11 – Dec 1	2.7–2.12
5	Class Creation: Writing Your Own Types	13	Dec 2 – Jan 5	3.1–3.9
6	Data Collections I: 1D Arrays & ArrayLists	12	Feb 1 – Feb 23	4.1–4.10
7	Data Collections II: 2D Arrays, Search, Sort, Recursion	8	Feb 24 – Mar 10	4.11–4.17
8	Ship It: GUI & Final Build	5	Mar 16 – Mar 25	Synthesis, Practices 1–4
9	Mock Exams & Presentations	3	Mar 29 – Mar 31	Practices 1–5

Full session-by-session dates are in **Pacing-Map-80-Sessions.md**.

5. What a Class Session Looks Like

Every session follows the same 45-minute shape so you always know what to expect.

Time	Segment	What happens
0:00–0:05	Do Now	A short prompt on the board. Predict output, spot the bug, or answer one AP-style question. Graded as participation.
0:05–0:12	Debrief & Objective	We review the Do Now and I state the day's objective in one sentence.
0:12–0:25	Direct Instruction / Live Code	I write code on the projector. You follow along in Replit. Mistakes are made on purpose.
0:25–0:40	Lab Work	You work the day's exercises from the GitHub repo, alone or in pairs. I circulate.
0:40–0:45	Exit Ticket & Assignment	One question that proves you got it, plus the homework posted in Google Classroom.

6. The GitHub Workflow

Every assignment in this course lives in a **GitHub repository**. Not a Google Doc. Not a PDF. A repo.

This applies to written work too. If I ask you to explain the difference between `==` and `.equals()`, you answer it in a Markdown file, in a repo, committed with a message.

The five-step routine

1. **Fork** the assignment repo from the class GitHub organization to your own account.
2. **Import** your fork into Replit (Replit → Create → Import from GitHub).
3. **Work** — write your Java in `src/`, write your answers in the `exercises/` Markdown files.
4. **Commit and push** with a clear message: `git commit -m "Complete exercises 1-4, add Rectangle class"`.
5. **Submit** — paste the URL of *your fork* into the Google Classroom assignment and click Turn In.

Non-negotiable repo standards

Starting Session 5, these are graded on every submission:

- ☐ `README.md` at the root, filled in with your name, the assignment, and a short summary of what you built
- ☐ At least **three separate commits** with meaningful messages — not one giant `"done"` commit
- ☐ No files named `Untitled.java`, `test2.java`, or `finalFINAL.java`
- ☐ Code is indented consistently and uses descriptive variable names
- ☐ Every class and every non-trivial method has a comment explaining its purpose

Commit history is part of the grade because *how* you work is part of the job.

7. Grading

Category	Weight	What's in it
Participation	10%	Do Nows, exit tickets, contributing in discussion, engaging in peer code review, arriving ready to work
Classwork	25%	In-session lab exercises pushed to GitHub by end of period or start of next
Homework	15%	Take-home exercise sets and readings, typically 20–30 minutes, 1–2 per week
Quizzes & Tests	25%	Short quizzes (~15 min) roughly every two weeks; a unit test at the end of each unit; two full mock AP exams
Year-Long Project	25%	Six graded milestones plus a final build and presentation
100%		

Grade scale

Letter	Range	Letter	Range
A+	97–100	C+	77–79
A	93–96	C	73–76
A–	90–92	C–	70–72
B+	87–89	D	65–69
B	83–86	F	below 65
B–	80–82		

How coding assignments are scored

Every coding assignment is out of 100 and scored on the same five dimensions, so you always know what I'm looking for:

Dimension	Points	Question it answers
Correctness	40	Does it compile, run, and produce the required output?
Code quality	20	Is it readable? Good names, consistent indentation, no dead code?
Documentation	15	Comments where they're needed, and a real README?
Problem solving	15	Did you choose a sensible approach, or brute-force it?
Git practice	10	Multiple meaningful commits, clean repo, correct submission?

8. Course Policies

Late work

- **Within 1 session late:** -10%
- **Within 2 sessions late:** -25%
- **After 2 sessions:** graded for feedback only, maximum 50%
- **Unit tests and project milestones** must be made up within one week; talk to me *before* the due date if something is going on. I am far more flexible with people who tell me in advance than with people who tell me after.

Attendance

This class meets 80 times. Missing three sessions means missing nearly 4% of the course. If you are absent, the lesson plan and all materials are in Google Classroom the same day — catch up before the next session, and come to me with specific questions.

Academic integrity and AI

You will use AI tools in your career. You need to learn to use them as a professional, not as a shortcut, so here are the actual rules:

Allowed, always: - Asking AI to explain a concept you don't understand - Asking AI to explain what a compiler error message means - Using AI to review code *you already wrote* and suggest improvements - Using autocomplete for boilerplate you already understand

Allowed with citation — add a comment in your code saying what you used and why: - Getting AI help debugging a specific problem - Asking AI for an example of a syntax pattern

Not allowed: - Submitting AI-generated code you cannot explain line by line - Using AI on quizzes, tests, or mock exams - Copying a classmate's code, with or without permission

The enforcement mechanism is simple: I may ask you at any time to walk me through any line of any assignment you submitted. If you cannot explain your own code, the grade for that assignment becomes zero regardless of whether it works. This is also exactly what happens in a technical job interview.

On collaboration: talking through an approach with a classmate is encouraged and is how real engineering works. Typing into each other's keyboards is not. The rule of thumb: leave the conversation and write the code yourself.

9. The Year-Long Project (25%)

Starting in Week 3, you will design and build **one original Java desktop application with a graphical user interface** that grows all year. Every unit you learn gets applied to it.

Milestone	Due session	Date	Weight	Deliverable
M1 — Proposal	S12	Thu Oct 8	10%	Problem statement, target user, feature list, mockup sketch
M2 — Core logic	S39	Tue Dec 1	15%	Working console version using conditionals and loops
M3 — Class design	S52	Tue Jan 5	20%	Full class hierarchy with constructors, accessors, mutators
M4 — Data layer	S64	Tue Feb 23	20%	Arrays/ArrayLists holding real data, with search and sort
M5 — GUI build	S75	Thu Mar 18	20%	Swing interface wired to your logic
M6 — Ship & present	S79-80	Mar 30-31	15%	Final repo, README with screenshots, 5-minute live demo

The January gap is project time. There is no class between Jan 6 and Jan 31. M3 is due Jan 5 precisely so that gap is free — but strong students use it to get a head start on M4.

Full specification, example project ideas, and rubrics are in **Year-Long-Project-Guide.md**.

10. Getting Help

- **In class:** raise your hand. Every single question you have is a question three other people also have.
- **Email:** afarhadi_te@flatbush.org — I aim to reply within 24 hours on school days.
- **Google Classroom:** post in the class stream. Answering a classmate's question there counts toward participation.
- **Before or after class:** find me. Ten minutes in person beats an hour of frustration at home.

If you are lost, tell me while you are a little lost. Do not wait until you are very lost. Nobody in this room has ever gotten in trouble for asking a question.

11. Materials Checklist

- ☐ Laptop, charged, every session
 ☐ Google Classroom joined
 ☐ GitHub account created and username submitted to me
 ☐ Replit account created (signed in with GitHub)
 ☐ Code.org CSA section joined
 ☐ AP Classroom section joined
 ☐ Notebook or digital notes for hand-tracing code (you will trace code by hand on the AP exam — practice it all year)

Syllabus subject to adjustment. Any change to dates, weights, or policies will be announced in Google Classroom and this document will be updated with a revision date.

Revision 1.0 — September 8, 2026